

API Contract

The boundary between the two halves of the system. Both teams build against this document, so agree it before either side starts.

All paths are prefixed `/api/v1`. Every endpoint except `/auth/register` and `/auth/login` requires authentication and is implicitly scoped to the caller's `organization_id` — that scoping happens in the repository layer, not per-endpoint, so it cannot be forgotten.

Error envelope

Every failure — validation, business rule, or crash — returns the same shape:

```
{
  "error": {
    "code": "odoo_unavailable",
    "message": "Odoo is unreachable.",
    "details": null
  },
  "request_id": "8f2c1e4a9b..."
}
```

code	HTTP	Meaning
<code>validation_error</code>	422	Request body failed validation; <code>details</code> carries FastAPI's field errors
<code>unauthenticated</code>	401	Missing, expired or malformed token
<code>forbidden</code>	403	Authenticated but not permitted — never retry, never refresh
<code>not_found</code>	404	Resource absent, or belongs to another organization
<code>conflict</code>	409	State conflict, e.g. confirming an already-pushed invoice
<code>upload_rejected</code>	422	Bad MIME type or oversized file
<code>odoo_not_configured</code>	400	Organization has not connected Odoo yet
<code>odoo_auth_failed</code>	502	Credentials rejected by Odoo
<code>odoo_unavailable</code>	503	Odoo unreachable — transient, safe to retry
<code>ocr_failed</code> / <code>ocr_extraction_failed</code>	502	Mistral call failed, or produced nothing usable
<code>internal_error</code>	500	Unhandled; <code>request_id</code> is the log correlation key

The distinction between 401 and 403 matters to the frontend: 401 triggers a token refresh, 403 must not. Refreshing on a permissions error masks the real bug as a session problem.

Endpoints

Auth

Method	Path	Request	Response	Notes
POST	/auth/register	RegisterRequest	TokenResponse	Creates <code>Organization</code> + owner <code>User</code> in one transaction. Gate behind an invite code in production.
POST	/auth/login	OAuth2PasswordRequestForm (<code>username</code> = email)	TokenResponse	Argon2 verify; updates <code>last_login_at</code> . Always hashes a dummy on unknown email so timing is constant.
POST	/auth/refresh	RefreshRequest	TokenResponse	Refresh JWTs carry <code>typ=refresh</code> ; access tokens are rejected here.
GET	/auth/me	—	UserRead	Embeds <code>OrganizationRead</code> . Frontend bootstrap.
POST	/auth/change-password	ChangePasswordRequest	204	

Organization & Odoo connection

Method	Path	Request	Response
GET	/organization	—	OrganizationRead — never returns the API key
PUT	/organization/odoo	OdooCredentialsUpdate	OdooConnectionStatusRead — encrypts with Fernet, immediately calls <code>test_connection</code> , persists the resulting status
POST	/organization/odoo/test	—	OdooConnectionStatusRead

Testing the connection on save rather than on first use means the user finds out their credentials are wrong while they are still looking at the credentials form.

Invoices — the core

Method	Path	Request	Response	Description
POST	<code>/invoices/upload</code>	<code>multipart/form-data</code> : <code>file</code> , optional <code>auto_match=true</code>	<code>InvoiceDetailResponse</code>	Validates MIME by magic bytes, enforces the size cap while streaming, hashes for duplicate detection, stores the blob, runs OCR → KB lookup → PO fetch → ranking. Returns the full detail including candidates.
GET	<code>/invoices</code>	query: <code>status</code> , <code>q</code> , <code>vendor</code> , <code>min_score</code> , <code>date_from</code> , <code>date_to</code> , <code>page</code> , <code>page_size</code> , <code>sort</code>	<code>Page[InvoiceListItem]</code>	The work queue.
GET	<code>/invoices/{id}</code>	—	<code>InvoiceDetailResponse</code>	Everything the verification screen needs.
GET	<code>/invoices/{id}/file</code>	—	<code>FileResponse (inline)</code>	Streams the original bytes for the left pane. Org-scoped path check prevents traversal.
POST	<code>/invoices/{id}/rematch</code>	<code>RematchRequest</code>	<code>MatchResult</code>	Re-runs matching after the user corrects a bad OCR field — without re-running OCR.
GET	<code>/invoices/{id}/candidates</code>	<code>limit</code> , <code>refresh</code>	<code>list[MatchCandidate]</code>	Cached snapshot; <code>refresh=true</code> re-pulls from Odoo.
POST	<code>/invoices/{id}/confirm</code>	<code>ConfirmMatchRequest</code>	<code>InvoiceDetailResponse</code>	Learns the alias, then pushes to Odoo. See the ordering notes in document 03.
POST	<code>/invoices/{id}/push</code>	—	<code>InvoiceDetailResponse</code>	Retries a <code>push_failed</code> row.
POST	<code>/invoices/{id}/reject</code>	<code>RejectRequest</code>	<code>InvoiceDetailResponse</code>	
DELETE	<code>/invoices/{id}</code>	—	<code>204</code>	Soft-delete for audit; the blob is removed.

`POST /invoices/upload` takes 5–20 seconds because OCR and the Odoo fetch run inline. Both the client timeout and any reverse-proxy timeout must accommodate that — a proxy defaulting to 60s while axios waits 120s produces a confusing 504. If you later move processing to a queue, this endpoint returns `202` with the row in `processing` and the client polls `GET /invoices/{id}`; the frontend hook already handles that state.

Odoo passthrough

Method	Path	Query	Response
GET	<code>/odoo/purchase-orders</code>	<code>partner_id</code> , <code>q</code> , <code>state</code> , <code>limit</code>	<code>list[OdooPurchaseOrder]</code> — powers manual PO search
GET	<code>/odoo/purchase-orders/{po_id}</code>	—	<code>OdooPurchaseOrder</code>
GET	<code>/odoo/partners</code>	<code>q</code> , <code>limit</code>	<code>list[OdooPartner]</code> — powers the vendor picker when teaching an alias
GET	<code>/odoo/health</code>	—	<code>OdooConnectionStatusRead</code>

Vendor knowledge base

Method	Path	Request	Response
GET	<code>/vendors/kb</code>	<code>q</code> , <code>partner_id</code> , <code>page</code> , <code>page_size</code>	<code>Page[VendorKBRead]</code>
POST	<code>/vendors/kb</code>	<code>VendorKBCreate</code>	<code>VendorKBRead</code> (upsert)
GET	<code>/vendors/kb/{id}</code>	—	<code>VendorKBRead</code>
PATCH	<code>/vendors/kb/{id}</code>	<code>VendorKBUpdate</code>	<code>VendorKBRead</code>
DELETE	<code>/vendors/kb/{id}</code>	—	<code>204</code>
POST	<code>/vendors/kb/resolve</code>	<code>{"vendor_name": "..."} </code>	<code>VendorKBRead</code> <code> null</code> — preview what lookup would return
POST	<code>/vendors/kb/import</code>	—	<code>{"imported": n}</code> — seed from <code>res.partner</code> where <code>supplier_rank > 0</code>

`/vendors/kb/resolve` exists for debugging. When a match goes wrong, the first question is always "did the KB fire, and on what?" — this answers it without reading logs.

Stats & health

Method	Path	Response
GET	<code>/stats/dashboard</code>	Counts by status, average score, auto-match rate, top vendors, 30-day trend
GET	<code>/health</code> (unprefixed)	Liveness
GET	<code>/health/ready</code>	Readiness — performs a DB round trip

Schemas

The Pydantic models and the TypeScript interfaces are shown side by side. **Keep them in lockstep.** Once the backend exists, generate the TS types from `/openapi.json` so drift becomes a compile error instead of a runtime surprise.

Extracted invoice (OCR output)

This Pydantic model does double duty: it is the API response shape **and** the JSON Schema sent to Mistral as `document_annotation_format`. The field descriptions are prompt engineering — write them as instructions to the model, not as notes to yourself.

```
# app/schemas/ocr.py
from __future__ import annotations

from pydantic import BaseModel, ConfigDict, Field, field_validator

class ExtractedLineItem(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    description: str = Field(
        description="Full product or service description text of this line."
    )
    quantity: float | None = Field(
        default=None, description="Quantity billed. Null if absent."
    )
    unit_price: float | None = Field(
        default=None, description="Price per unit excluding tax."
    )
    line_total: float | None = Field(
        default=None, description="Extended amount for this line excluding tax."
    )
    product_code: str | None = Field(
        default=None, description="Vendor SKU / part number if printed."
    )
    unit: str | None = Field(
        default=None, description="Unit of measure, e.g. 'pcs', 'hrs', 'kg'."
    )

class ExtractedInvoice(BaseModel):
    """Sent to Mistral as document_annotation_format (json_schema)."""
    model_config = ConfigDict(populate_by_name=True)

    vendor_name: str | None = Field(
        default=None,
        description=(
            "Legal name of the SUPPLIER issuing this invoice (the party to be "
            "paid), not the customer/bill-to party."
        ),
    )
    vendor_tax_id: str | None = Field(
        default=None, description="Supplier VAT / GST / NTN / tax number."
    )
    vendor_address: str | None = Field(
        default=None, description="Supplier full address, single line."
    )
    invoice_number: str | None = Field(
        default=None, description="The invoice's own document number."
    )
    invoice_date: str | None = Field(
        default=None, description="Invoice issue date, ISO 8601 YYYY-MM-DD."
    )
    due_date: str | None = Field(
        default=None, description="Payment due date, ISO 8601 YYYY-MM-DD."
    )
    purchase_order_reference: str | None = Field(
        default=None,
```

```

        description=(
            "Any purchase order number referenced on the invoice, e.g. 'P000042'. "
            "Look for labels: PO, P.O., Order No, Your Order, Customer Order."
        ),
    )
    currency: str | None = Field(
        default=None, description="ISO 4217 code, e.g. USD, EUR, PKR, AED."
    )
    subtotal: float | None = Field(default=None, description="Net total excluding tax.")
    tax_amount: float | None = Field(
        default=None, description="Total tax/VAT/GST charged."
    )
    total_amount: float | None = Field(
        default=None, description="Grand total payable including tax."
    )
    line_items: list[ExtractedLineItem] = Field(
        default_factory=list, description="Every billed line."
    )

    @field_validator("currency")
    @classmethod
    def _upper_currency(cls, v: str | None) -> str | None:
        return v.upper()[:3] if v else None

```

```

// client/src/types/invoice.ts
export interface LineItem {
    id: string;
    line_number: number | null;
    description: string | null;
    sku: string | null;
    quantity: number | null;
    unit_price: number | null;
    line_total: number | null;
    tax_rate: number | null;
    /** 0..1 per-line OCR confidence; null when the model gave none. */
    ocr_confidence: number | null;
}

export interface ExtractedInvoice {
    id: string;
    /** Raw vendor string exactly as OCR'd - this is what gets aliased. */
    vendor_name: string | null;
    vendor_tax_id: string | null;
    vendor_address: string | null;
    invoice_number: string | null;
    /** ISO-8601 date (YYYY-MM-DD). */
    invoice_date: string | null;
    due_date: string | null;
    po_number: string | null;
    currency: string | null;
    subtotal: number | null;
    tax_amount: number | null;
    total_amount: number | null;
    line_items: LineItem[];
    ocr_confidence: number | null;
    raw_text: string | null;
}

```

Odoo objects

```
# app/schemas/odoo.py
class OdooPOLine(BaseModel):
    id: int
    name: str # description
    product_id: int | None = None
    product_name: str | None = None
    product_code: str | None = None
    product_qty: float = 0.0
    qty_received: float = 0.0
    qty_invoiced: float = 0.0
    price_unit: Decimal = Decimal("0")
    price_subtotal: Decimal = Decimal("0")
    price_total: Decimal = Decimal("0")
    product_uom_name: str | None = None

class OdooPurchaseOrder(BaseModel):
    id: int
    name: str # 'PO00042'
    partner_id: int
    partner_name: str
    partner_ref: str | None = None # the vendor's own reference on the PO
    date_order: dt.datetime | None = None
    date_planned: dt.datetime | None = None
    state: str # draft | sent | to approve | purchase | done | cancel
    invoice_status: str | None = None # no | to invoice | invoiced
    currency_id: int | None = None
    currency_name: str | None = None
    amount_untaxed: Decimal = Decimal("0")
    amount_tax: Decimal = Decimal("0")
    amount_total: Decimal = Decimal("0")
    order_line: list[OdooPOLine] = []

class OdooPartner(BaseModel):
    id: int
    name: str
    vat: str | None = None
    email: str | None = None
    supplier_rank: int = 0
```

```
export interface PurchaseOrderLine {
    id: number;
    product_id: number | null;
    product_name: string;
    product_code: string | null;
    quantity: number;
    qty_received: number | null;
    qty_invoiced: number | null;
    price_unit: number;
    price_subtotal: number;
    taxes: string[];
}

export interface PurchaseOrder {
    /** Odoo `purchase.order` database id. */
    id: number;
    name: string; // e.g. "PO00042"
    partner_id: number;
    partner_name: string;
    partner_vat: string | null;
    date_order: string; // ISO-8601 datetime
    currency: string;
    amount_untaxed: number;
    amount_tax: number;
    amount_total: number;
    state: string;
    invoice_status: string;
    lines: PurchaseOrderLine[];
}
```

Decimal on the wire. Pydantic serializes `Decimal` to a JSON number by default, which JavaScript parses as a float. For AP amounts under 2^{53} that is lossless, so it is acceptable — but never do arithmetic on these in the browser. Display them, and let the backend compute every total and delta.

Scoring

```
# app/schemas/matching.py
class ComponentScore(BaseModel):
    name: str
    raw: float = Field(ge=0, le=100, description="0-100 for this component alone.")
    weight: float = Field(ge=0)
    weighted: float = Field(description="raw * weight / 100.")
    applicable: bool = True
    detail: str = ""

class ScoreBreakdown(BaseModel):
    components: list[ComponentScore]
    total: float = Field(ge=0, le=100)
    applied_weight_total: float
    method: MatchMethod = MatchMethod.FUZZY

class MatchCandidate(BaseModel):
    purchase_order: OdooPurchaseOrder
    score: float
    breakdown: ScoreBreakdown
    band: str # "high" | "medium" | "low"

class MatchResult(BaseModel):
    candidates: list[MatchCandidate]
    best: MatchCandidate | None = None
    kb_partner_id: int | None = None
    kb_hit: bool = False
    duration_ms: int = 0
```



```

export type ConfidenceTier = 'high' | 'medium' | 'low';
export type FieldMatchState = 'match' | 'mismatch' | 'partial' | 'missing';
export type ScoreComponentKey =
  | 'vendor_name' | 'po_number' | 'total_amount'
  | 'invoice_date' | 'line_items' | 'currency';

export interface ScoreComponent {
  key: ScoreComponentKey;
  label: string;
  /** 0..100 for this component alone. */
  score: number;
  /** 0..1, sums to 1 across all components. */
  weight: number;
  /** Precomputed by the backend so the UI never re-derives it. */
  weighted_score: number;
  /** Human-readable justification, shown in the breakdown panel. */
  rationale: string | null;
  invoice_value: string | null;
  po_value: string | null;
  state: FieldMatchState;
}

export interface ScoreBreakdown {
  /** 0..100 overall. Equals sum(components[].weighted_score), rounded. */
  total_score: number;
  tier: ConfidenceTier;
  components: ScoreComponent[];
  /** True when a learned vendor alias contributed to this match. */
  alias_applied: boolean;
  matched_alias_id: string | null;
}

export interface POCandidate {
  purchase_order: PurchaseOrder;
  score: ScoreBreakdown;
  /** 1-based position in the ranked list. */
  rank: number;
}

```

The backend sends `weighted_score` and `tier` pre-computed rather than letting the UI derive them. Two implementations of the same threshold logic will eventually disagree, and when they do, the number on screen stops matching the number in the audit log.

Match records

```
/** Row shape for the dashboard list - deliberately lighter than MatchDetail. */
export interface MatchHistory {
  id: string;
  file_name: string;
  file_size: number;
  mime_type: string;
  status: MatchStatus;
  vendor_name: string | null;
  invoice_number: string | null;
  invoice_date: string | null;
  total_amount: number | null;
  currency: string | null;
  matched_po_name: string | null;
  confidence_score: number | null;
  tier: ConfidenceTier | null;
  created_at: string;
  updated_at: string;
  confirmed_at: string | null;
  odoo_bill_id: number | null;
  error_message: string | null;
}

/** Full payload for the verification screen. */
export interface MatchDetail extends MatchHistory {
  extracted_invoice: ExtractedInvoice;
  candidates: POCandidate[];
  /** Odoo id of the currently selected PO; null until a human picks one. */
  selected_po_id: number | null;
  /** Server-relative URL for the original document bytes. */
  file_url: string;
  page_count: number | null;
}

export type MatchResult = MatchDetail;

export type MatchStatus =
  | 'uploaded' | 'processing' | 'ocr_failed' | 'pending'
  | 'confirmed' | 'rejected' | 'pushing' | 'pushed' | 'push_failed';
```

Keeping the list row lighter than the detail matters: `MatchDetail` embeds every candidate PO with all its lines, which is tens of kilobytes. Sending that for fifty dashboard rows would make the list view slower than the screen it feeds into.

Confirmation

```
/** Only fields the user actually edited are sent, hence Partial. */
export type InvoiceFieldCorrections = Partial<
  Pick<
    ExtractedInvoice,
    | 'vendor_name' | 'invoice_number' | 'invoice_date' | 'due_date'
    | 'po_number' | 'currency' | 'subtotal' | 'tax_amount' | 'total_amount'
  >
>;

export interface ConfirmMatchRequest {
  /** Odoo purchase.order id the human is committing to. */
  purchase_order_id: number;
  corrections: InvoiceFieldCorrections;
  /** Persist vendor_name -> partner_id into the knowledge base. */
  learn_vendor_alias: boolean;
  /** Create the vendor bill in Odoo (vs. only recording the match). */
  push_to_odoo: boolean;
  notes: string | null;
}

export interface ConfirmMatchResponse {
  match: MatchHistory;
  odoo_bill_id: number | null;
  odoo_bill_name: string | null;
  created_alias: VendorAlias | null;
}
```

Sending only the changed fields — not the whole object — is what keeps the `user_corrections` audit trail honest. If the client posted every field, every confirmation would look like the user rewrote the entire invoice.

The backend's equivalent adds one field the TypeScript omits, because the frontend does not expose it in v1:

```
class ConfirmMatchRequest(BaseModel):
    odoo_po_id: int
    corrections: list[FieldCorrection] = []
    learn_alias: bool = True
    push_action: Literal["create_bill", "note_only", "none"] = "create_bill"
    post_bill: bool = False # leave False: posting makes a live journal entry
```

`push_action` lets an organization start in `note_only` mode while matching accuracy is still being proven, then switch to `create_bill` once they trust it.

Knowledge base

```
export interface VendorAlias {
  id: string;
  /** Normalized OCR vendor string, e.g. "acme corp ltd". */
  alias: string;
  /** Verbatim string first seen on an invoice. */
  raw_value: string;
  odoo_partner_id: number;
  odoo_partner_name: string;
  /** How many confirmations have reinforced this mapping. */
  hit_count: number;
  /** 0..1 - grows with hit_count; used to auto-apply above a threshold. */
  confidence: number;
  created_by: string | null;
  created_at: string;
  updated_at: string;
  last_used_at: string | null;
}
```

Pagination

```
# app/schemas/common.py
class Page(BaseModel, Generic[T]):
    items: list[T]
    total: int
    page: int
    page_size: int
    pages: int
```

```
export interface Paginated<T> {
  items: T[];
  total: number;
  page: number;
  page_size: number;
  pages: number;
}

/** FastAPI's default validation error shape. */
export interface FastAPIValidationError {
  loc: (string | number)[];
  msg: string;
  type: string;
}

export interface ApiErrorBody {
  detail: string | FastAPIValidationError[];
}
```

Keeping the two sides in sync

Hand-written types drift. The moment the backend is running, replace `client/src/types/*.ts` with generated output:

```
npx openapi-typescript http://localhost:8000/api/v1/openapi.json -o src/types/api-generated.ts
```

Until then, treat this document as the contract and review changes to it as carefully as changes to code — a silent rename here surfaces as a `undefined is not an object` three screens away.